

Learning Memory Access Patterns

Milad Hashemi, Kevin Swersky, Jamie A. Smith, Grant Ayers, Heiner Litz,
Jichuan Chang, Christos Kozyrakis, Parthasarathy Ranganathan

Presenter: Jay Dasari

Background/Motivation

- Von Neumann Bottleneck/Memory Wall
 - Significant gap in space and speed between CPU and DRAM
 - “over 50% of their compute cycles waiting for data”
 - Typically addressed through memory hierarchies
- Hardware Limitations of Lookup Tables
 - Cloud application working sets larger than lookup table sizes
 - “prediction accuracy drops sharply when the working set is larger than the predictive table”
 - Scaling is expensive

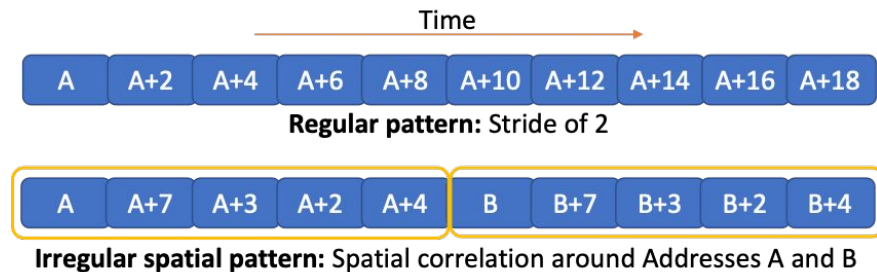
Current State of Prefetchers

- **Stride Prefetchers**

- Lock onto repeated fixed deltas
- Common in modern processors, relatively easy to implement
- High recall in predictable patterns, but struggle with irregular or non-linear patterns

- **Correlation Prefetchers**

- Designed to learn irregular patterns that repeat over time
- Store past memory accesses in large lookup tables
- Large and costly, not typically in modern multi-core processors



How can we utilize Neural Nets to improve prefetching?

Reframing the Problem

Hardware Architecture

Natural Language Processing

Memory Addresses



Words in Vocabulary

Programs



Sentences

Correlation Prefetchers



N-gram Language Models

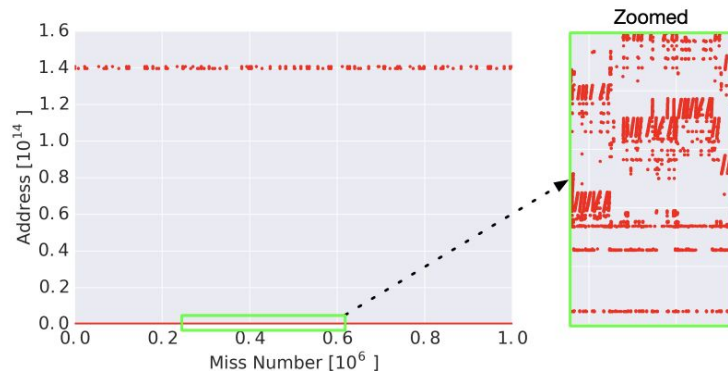
Neural Prefetching



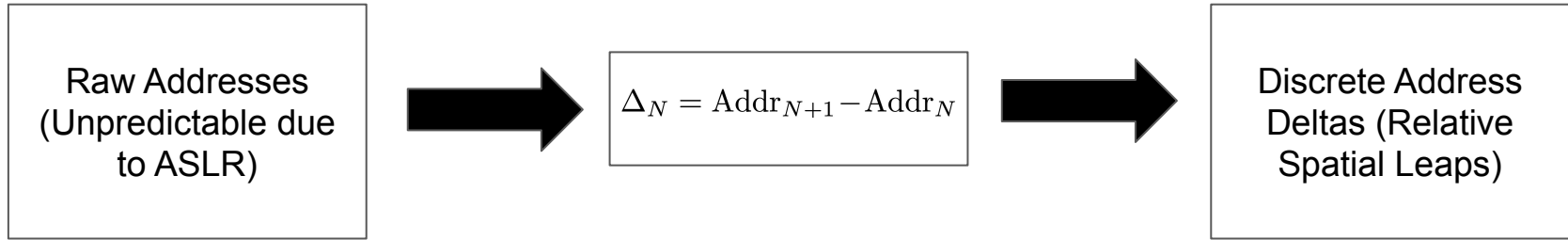
Recurrent Neural Networks
(Specifically LSTMs)

Problems with Using RNNs

- Extremely large but sparse address space
 - Out of 2^{64} address space, only 10M unique cache miss addresses appeared from a 100M miss training set
 - Normalizing scattered data into finite precision FP numbers destroys exact precision and accuracy
- Address Space Layout Randomization
 - Randomizes a program's code and data memory locations
 - Difficult for RNN to predict exact memory locations, as locations change per run



Solution: Shrinking the Address Space



From Regression to Classification

- Model treats address space as discrete vocabulary
- Performs next-word classification rather than predicting exact locations

Bypasses ASLR

- Relative spatial movements (deltas) stay consistent across runs

Key Insight: Moving from absolute addresses to deltas shrinks the vocabulary required for 50% coverage in the gems dataset from 4,280,000 to just 18

Models

- Embedding LSTM
 - 2-layer LSTM
 - Classifies over a vocabulary of the 50,000 most frequent deltas observed in training
- Clustering + LSTM
 - Divides memory into separate regions
 - Tries to exploit “local context”
 - Typically smaller models

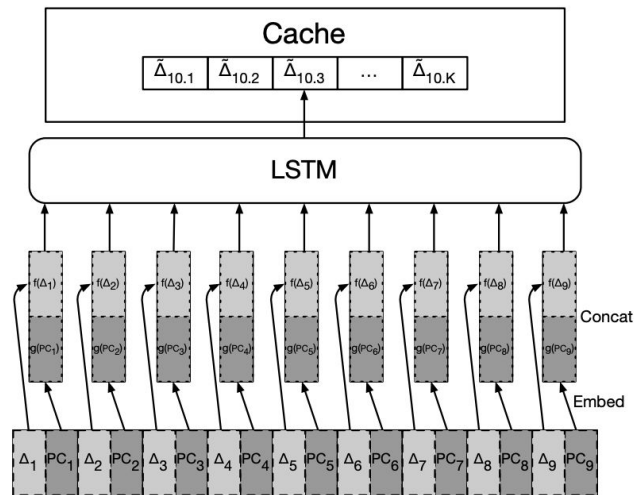
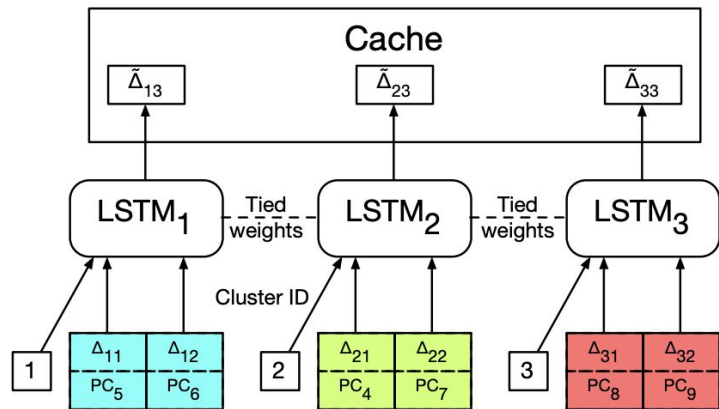
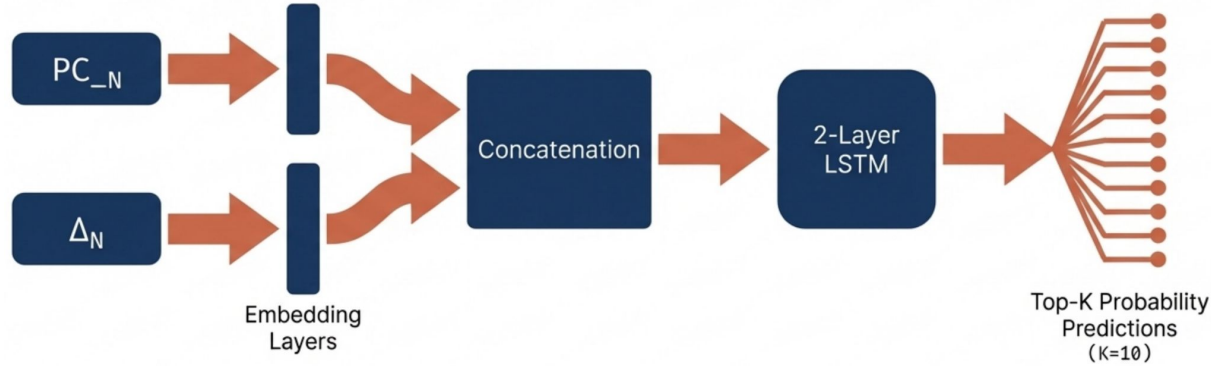


Figure 2. The embedding LSTM model. f and g represent embedding functions.



(b) The clustering + LSTM model.

Embedding LSTM



- Inputs

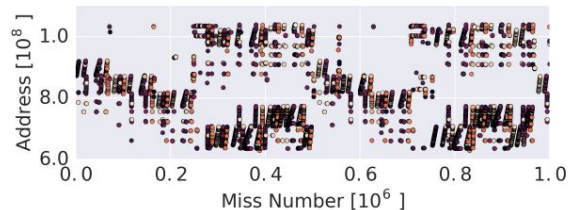
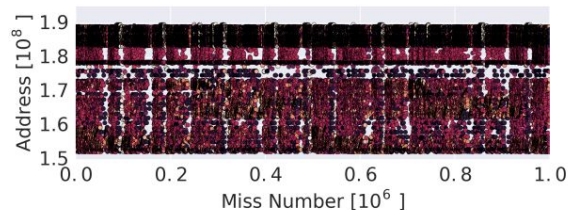
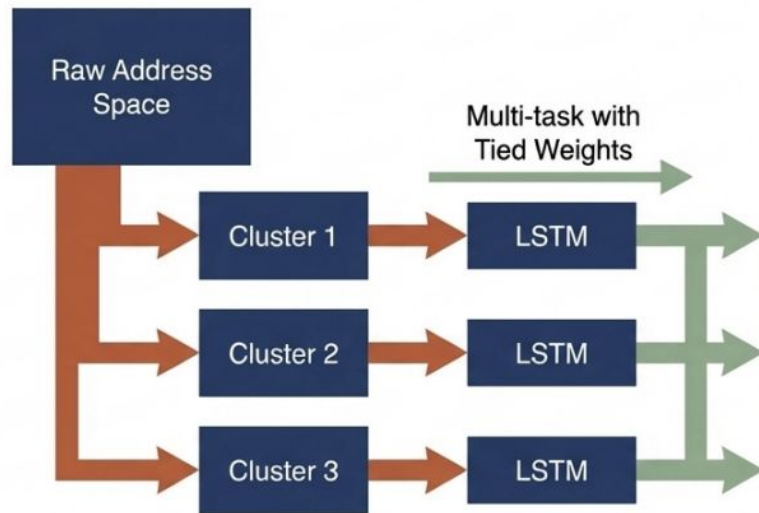
- Program Counter (PC)
- Delta

- Process

- Utilizes most frequent 50,000 deltas to predict top-10 most likely memory addresses of next access

Clustering + LSTM

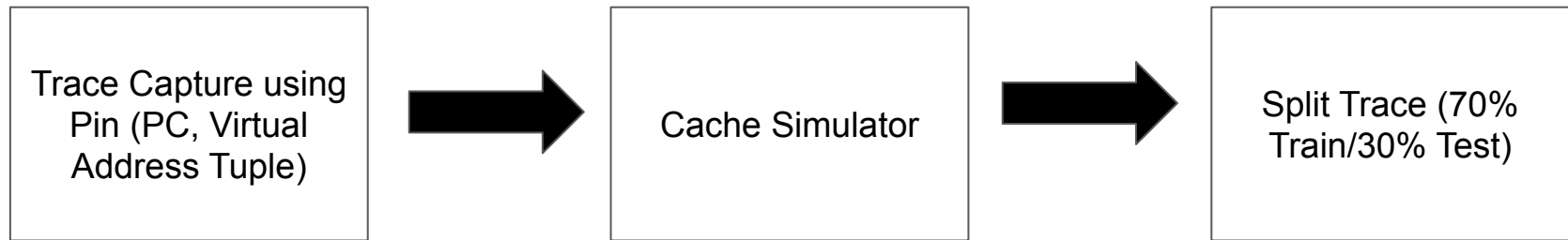
- Partition Clusters using K-means clustering
- Inputs
 - Normalized Deltas (saves memory)
 - Possible as addresses usually have similar magnitudes
 - Cluster ID
 - Effectively gives each cluster stream a unique set of biases



Benefits/Drawbacks

- Embedding
 - Benefits
 - Increased breadth of context (global and local)
 - Semantic learning (implicitly identifying loop unrolling/linked-list traversal)
 - Drawbacks
 - Accuracy Ceiling
 - Increased model compute and memory footprint
- Cluster + LSTM
 - Benefits
 - Higher efficiency
 - Superior recall
 - Drawbacks
 - Additional preprocessing
 - Increased risk of inaccuracies when denormalized

Evaluating the Models



Datasets:

- SPEC CPU2006: Academic memory-intensive standards
- Google Web Search: Enterprise-scale complexity and irregularity

Hardware Baselines:

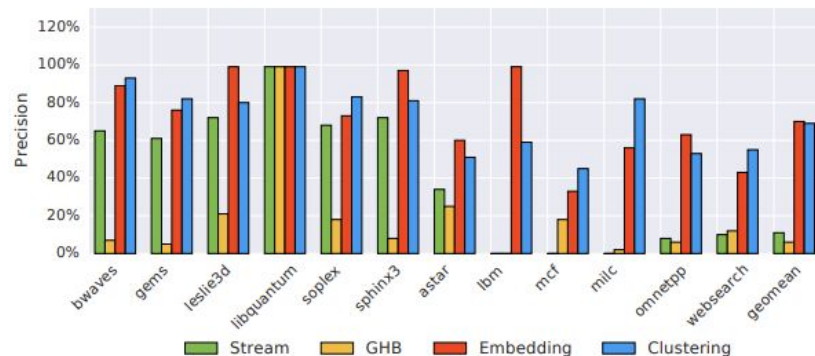
- 10-stream Stream Prefetcher
- GHB Correlation Prefetcher

Metrics:

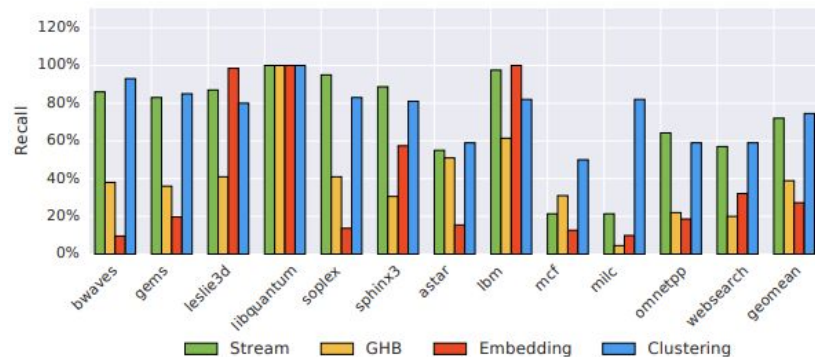
- Precision-at-10: Accuracy of the top-10 predictions generated
- Recall-at-10: Total diversity and coverage of predictions

Results

- Overall, the LSTM models perform significantly better than traditional methods
- Most significant performance gap is in the Google Web Search, where typical methods fail



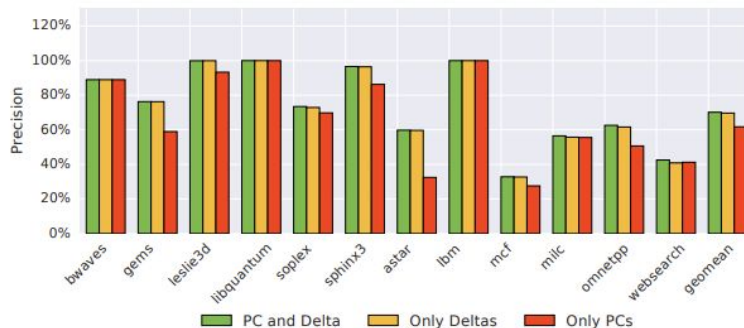
(a) Precision



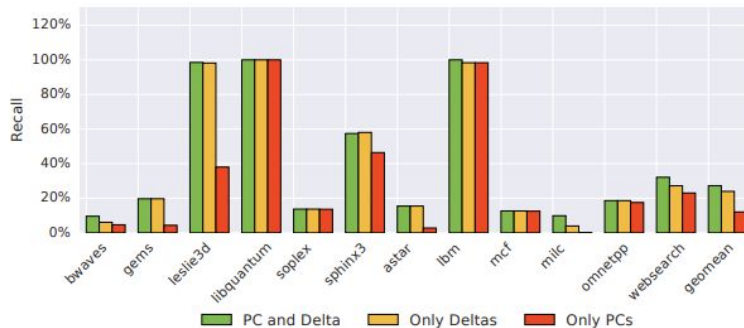
(b) Recall

Embedded LSTM with 1 Input vs Both

- Observed that both PC and Deltas had a good amount of predictive info
- Majority of info contained within Delta sequence
- However, PC sequence helps improve recall overall



(a) Precision



(b) Recall

Interpreting Program Semantics

- Models can then be introspected to gain insights into data due to their ability to learn patterns
- For example, one cluster shows patterns of loop unrolling, while another shows repeated pointer dereferences

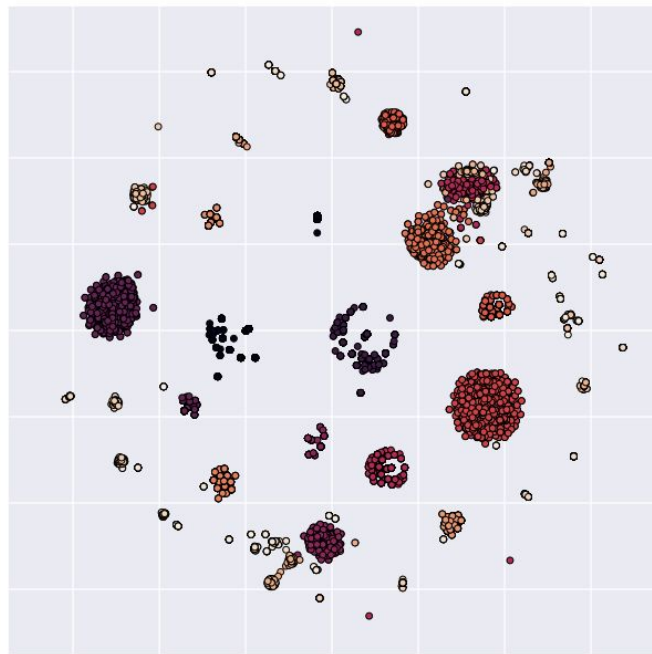


Figure 7. A t-SNE visualization of the concatenated (Δ, PC) embeddings on *mcf* colored according to PC instruction.

Future Considerations

1.) Memory to Compute Shift

NNs compress learned representations, shifting hardware prediction from a near-impossible memory problem to a scalable compute problem

2.) Reinforcement Learning

Future environments will shift to RL to optimize models directly for execution time and cache hits online

3.) Security Implications

Understanding and securing learned predictive AI systems is the next critical frontier, as previous attacks such as spectre and meltdown cannot be repeated

Thank You